Université Privée de Fès
Faculté des Sciences de l'Ingénieur

Système de surveillance des angles morts

Cahier des charges et rapport de réalisation (maquette)

Groupe : Anas Bitta et Mohamed ElBouzaydy

Encadrant : Mme IMANE HALKHAMS

Filière : Génie des Systèmes Embarqués (UPF)

Année : 2025/2026

Ce document décrit le cahier des charges et la réalisation d'une maquette de Système de surveillance des angles morts, basée sur caméras, capteurs ultrasoniques et interface Qt.

Table des matières

Résumé **v**

Abstract **vi**

1 Introduction **1**

2 Cahier des charges **2**

2.1 Contexte et parties prenantes 2

2.2 Objectifs 2

2.3 Exigences fonctionnelles 2

2.3.1 Entrées 2

2.3.2 Sorties 2

2.3.3 Scénarios d'utilisation 3

3 Exigences techniques **4**

3.1 Plateforme matérielle 4

3.2 Plateforme logicielle 4

3.3 Contraintes temps réel 4

4 Réalisation et implémentation **5**

4.1 Architecture générale 5

4.2 Schéma matériel et câblage 5

4.3 Description de l'algorithme fonctionnel 6

4.4 Problèmes rencontrés et solutions 6

5 Tests et validation **7**

5.1 Plan de tests 7

5.2 Critères d'acceptation 7

5.3 Résultats attendus 7

6 Gestion de projet **8**

6.1 Planning (Gantt simplifié) 8

6.2 Répartition des tâches 8

7 Bilan et perspectives **9**
7.1 Bilan 9
7.2 Améliorations possibles 9

A Annexes **10**
A.1 Schémas électriques 10
A.2 Liste du matériel 10
A.3 Fiches techniques 10

Bibliographie **11**

Table des figures

Liste des tableaux

6.1 Planning prévisionnel (à adapter). 8

A.1 Matériel principal utilisé pour la maquette. 10

Résumé

Ce projet consiste à concevoir une maquette d'un **système embarqué de surveillance des angles morts** pour un véhicule. Le prototype combine des caméras grand-angle et des capteurs ultrasoniques, traités par une unité embarquée (Raspberry Pi), et présente les informations sur une interface graphique développée sous Qt. En cas d'obstacle détecté lors de l'activation d'un clignotant, le système génère une alerte visuelle et sonore. Une fonction de sécurité supplémentaire verrouille automatiquement les portes si le véhicule est à l'arrêt et qu'un objet approche par l'arrière. Une étude d'intégration V2X est prévue si le temps le permet.

Mots-clés : angles morts, caméra, ultrason, Qt, V2X, ADAS.

Abstract

This project aims to build a small-scale prototype of a **blind-spot monitoring system**. The system combines wide-angle cameras and ultrasonic sensors processed by an embedded unit (Raspberry Pi) and displays data on a Qt graphical interface. When an obstacle is detected while the turn signal is active, visual and audible warnings are issued. An additional safety feature locks the doors when the vehicle is stopped and a rear-side object approaches. A V2X integration study is planned if time allows.

Chapitre 1

Introduction

La sécurité routière nécessite aujourd'hui des solutions embarquées pour assister le conducteur. Les collisions liées aux angles morts restent une cause importante d'accidents lors de changements de voie. Ce projet vise à réaliser une maquette fonctionnelle capable de montrer, lors de l'activation d'un clignotant, une vue directe de l'angle mort via une caméra dédiée et d'alerter le conducteur en cas de danger. Le prototype met l'accent sur la simplicité, la robustesse et la démonstration d'un concept applicable à des systèmes ADAS.

Chapitre 2

Cahier des charges

2.1 Contexte et parties prenantes

— **Contexte :** Développement d'un système embarqué pour la détection des angles morts appliqué à une maquette automobile.
— **Parties prenantes :** Équipe étudiante (développement), encadrant pédagogique, utilisateurs cibles (conducteurs), fournisseur capteurs.

2.2 Objectifs

— **Fonction principale :** Détecter et afficher les objets présents dans les angles morts et alerter le conducteur.
— **Fonctionnalités secondaires :** Affichage automatique selon clignotant, verrouillage portes à l'arrêt si anomalie arrière, collecte de logs, étude V2X.
— **Critères de performance :** temps de réponse $\leq 300\,\text{ms}$, précision distance $\pm 3\,\text{cm}$, flux vidéo $\geq$ 720p@30 fps.

2.3 Exigences fonctionnelles

2.3.1 Entrées

Caméras gauche/droite/arrière, capteurs ultrasoniques, signal clignotant (GPIO/CAN), signal vitesse (simulé).

2.3.2 Sorties

Interface Qt (vidéo + distances), buzzer/LED, commande relais pour verrouillage.

2.3.3 Scénarios d'utilisation

1. Clignotant gauche activé → affichage caméra gauche en grand + calcul de distance.
2. Clignotant droit activé → affichage caméra droite en grand + calcul de distance.
3. Véhicule à l'arrêt + détection arrière anormale → verrouillage portes + alerte.

Chapitre 3

Exigences techniques

3.1 Plateforme matérielle

— Carte : **Raspberry Pi 4 model B** (4 GB) ou ordinateur personnel.
— Caméras : modules USB/CSI, grand-angle 120–150°, 720p.
— Capteurs : HC-SR04 ou équivalents industriels.
— Alimentation : entrée 12 V (simulée) → régulateur 5 V pour alimentation Pi et périphériques.
— Actionneurs : relais 5 V pour verrouillage, buzzer.

3.2 Plateforme logicielle

— OS : Raspberry Pi OS (Linux) - windows si on travaille sur notre machine personnel.
— Langages : Qt (Qml) ; C++ (liaison capteur - interface), python, C.
— Frameworks : Qt / PyQt5, OpenCV.

3.3 Contraintes temps réel

Les boucles capteurs et affichage doivent être conçues pour respecter une latence maximale de 300 ms entre la détection et l'alerte.

Chapitre 4

Réalisation et implémentation

4.1 Architecture générale

L'architecture générale du système repose sur une approche modulaire, répartie en quatre blocs principaux, chacun jouant un rôle précis dans le fonctionnement global.

- **Bloc de perception** : constitué des caméras et des capteurs ultrasoniques, il collecte les données de l'environnement immédiat du véhicule (zones latérales et arrière).
- **Bloc de traitement embarqué** : basé sur la carte **Raspberry Pi 4** ou équivalent, il reçoit les données des capteurs, exécute les traitements de détection et applique les règles logiques de décision.
- **Bloc d'alerte et d'action** : gère les signaux visuels et sonores destinés au conducteur (interface Qt, LED, buzzer) et pilote les actionneurs tels que le relais de verrouillage des portes.
- **Bloc d'interface utilisateur** : affiche en temps réel les flux vidéo et les distances détectées sur une interface Qt ergonomique et futuriste, tout en signalant les dangers.

L'échange de données entre ces blocs s'effectue via des protocoles simples (GPIO, I2C, USB) et repose sur un traitement synchrone permettant une réaction rapide (moins de 300 ms).

```
[Caméras / Capteurs] → [Traitement embarqué (Raspberry Pi)] → [Interface
Qt + Alertes] → [Actionneurs (Relais, LEDs, Buzzer)]
```

4.2 Schéma matériel et câblage

En l'absence de schéma graphique définitif, la description suivante présente le câblage général prévu pour la maquette du système.

— Les **caméras gauche, droite et arrière** sont reliées à la carte Raspberry Pi via les ports USB (ou CSI selon le modèle).
— Les **capteurs ultrasoniques HC-SR04** sont connectés sur les broches GPIO (Trigger/Echo) avec résistances de protection.
— Le **buzzer** et les **LEDs d'alerte** sont pilotés par des sorties GPIO configurées en mode sortie.
— Le **relais de verrouillage** est commandé par une sortie GPIO via un transistor (2N2222 ou équivalent).
— L'ensemble est alimenté par une **source 12 V simulée**, régulée à 5 V grâce à un module convertisseur DC/DC.

`[Caméras / Capteurs] → GPIO / USB → Raspberry Pi → [Qt Interface / Buzzer / Relais / LEDs]`

4.3 Description de l'algorithme fonctionnel

1. **Initialisation :** configuration GPIO, caméras et capteurs.
2. **Acquisition :** boucle continue d'acquisition images + distances.
3. **Analyse :** filtrage bruit, calcul distances.
4. **Décision :**
   — Clignotant gauche/droit → caméra correspondante active.
   — Si obstacle détecté → alerte visuelle + sonore.
   — Véhicule arrêté + objet arrière → verrouillage portes.
5. **Affichage Qt :** mise à jour flux vidéo, distance, alertes.

4.4 Problèmes rencontrés et solutions

— **Synchronisation :** threads + buffer circulaire.
— **Bruit ultrasons :** filtre médian + moyenne glissante.
— **Latence vidéo :** réduction résolution + ROI.

Chapitre 5

Tests et validation

5.1 Plan de tests

— Test 1 : Mesure distances à 1 m, 2 m, 3 m.
— Test 2 : Activation clignotants, affichage correct.
— Test 3 : Véhicule arrêté + approche arrière → verrouillage portes.

5.2 Critères d'acceptation

— Temps de réaction < 300 ms.
— Taux de détection $\geq 90\%$.
— Stabilité 1h sans crash.

5.3 Résultats attendus

Remplir ici les tableaux de résultats expérimentaux après tests.

Chapitre 6

Gestion de projet

6.1 Planning (Gantt simplifié)

Phase	Durée / Remarques
Étude	Semaines 1–2 : choix matériel, cahier des charges
Montage matériel	Semaines 3–4 : câblage et intégration
Développement logiciel	Semaines 5–7 : traitement, UI
Tests	Semaine 8 : validation et mesures
Rédaction	Semaine 9 : rapport et soutenance

Table 6.1 – Planning prévisionnel (à adapter).

6.2 Répartition des tâches

Indiquer ici la répartition réelle entre membres (ex : Anas — interface Qt / Mohamed — électronique / etc.)

Chapitre 7

Bilan et perspectives

7.1 Bilan

Décrire ce qui a été réalisé et ce qui reste à améliorer.

7.2 Améliorations possibles

— IA embarquée (YOLO).
— Communication V2X.
— Optimisation temps réel.

Annexe A

Annexes

A.1 Schémas électriques

Insérer schémas détaillés (PNG/PDF).

A.2 Liste du matériel

Désignation	Référence / modèle	Qté
Raspberry Pi 4	4 GB	1
Caméra	720p	
Capteurs ultrasoniques	HC-SR04	4
Relais 5V	Module relais 1 canal	1
Buzzer	5V	1

Table A.1 – Matériel principal utilisé pour la maquette.

A.3 Fiches techniques

Inclure les datasheets des capteurs et caméras.

Bibliographie

Bibliographie

[1] OpenCV documentation, `https://opencv.org/`.

[2] Qt for Application Development, The Qt Company.

[3] Texas Instruments mmWave Sensors (pour lecture complémentaire).